

Display File Interpreter

We have seen that display file contains information necessary to construct the picture. This information is in the form of commands. The program which converts these commands into actual picture is called display file interpreter. It is an interface between graphics representation in the display file and the display device, as shown in the Fig. 1.26.

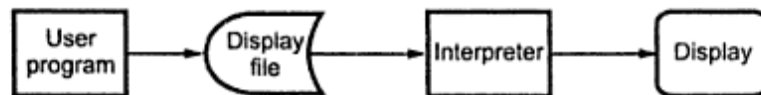


Fig. 1.26 Display file and interpreter

As shown the Fig. 1.26, the display process is divided into two steps : first the image is stored in the display file structure and then it is interpreted by an appropriate interpreter to get the actual image.

Instead of this, if we store actual image for particular display device it may not run on other displays. To achieve the device independence the image is stored in the raw format, i.e. in the display file format and then it is interpreted by an appropriate interpreter to run on required display.

Another advantage of using interpreter is that saving raw image takes much less storage than saving the picture itself.

1.3 Display File and its Structure

We know that in raster scan displays image information is stored in the frame buffer. It includes information of all pixels. On the other hand, the vector refresh displays store only the commands necessary for drawing the line segments. Here, input to the vector generator is stored instead of the output. The file used to store the commands necessary for drawing the line segments is called **display file**.

In vector refresh display system, display processor uses the information in the display file to draw lines with the help of vector generating algorithms. This is illustrated in Fig. 1.24. Therefore, we can say that display files provides an interface between the image specification process and the image display process. It also describes image in a compact format. The concept of display file may be applied to devices other than refresh displays. Such files are called **pseudo display files**, or **metafiles**.

The Fig. 1.24 shows the structure of display file. We know that it contains series of commands. Each display file command contains two fields, an **operation code (opcode)** and **operands**. Opcode identifies the command such as draw line, move cursor etc., and operand provides the coordinates of a point to process the command.

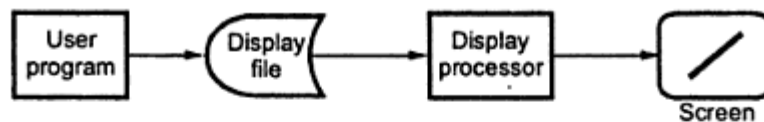


Fig. 1.24 Vector refresh display system

One way to store opcode and operands of series of commands is to use three separate arrays. One for operation code, one for operand 1, i.e. x coordinate and one for operand2, i.e. y coordinate. This is illustrated in Fig. 1.25. The display file stores all command to be needed to create a specified image. It is necessary to assign meaning to the possible operation codes before we can proceed to interpret them. For simplicity let us consider three commands MOVE, LINE and PLOT, and assign opcodes to these commands as shown in table 1.2.

Command	Opcode
MOVE	1
LINE	2
PLOT	3

Table 1.2

Once the opcodes are defined we can write commands needed to draw image into the display file. The following algorithm gives the steps required to insert command in the display file.

Algorithm : Insert Command in display file

INSERT_COMM (Opcode, X, Y)

Arguments Op, X, Y instruction fields

Global DF_Op, DF_X, DF_Y the three display file arrays

FREE the position of next freecell

Constant DFSIZE size of the display file

BEGIN

IF FREE > DFSIZE **THEN** RETURN

ERROR 'Display file is full'.

2. DF_OP [i] $\leftarrow$ Opcode;

DF_X [i] $\leftarrow$ X ;

DF_Y [i] $\leftarrow$ Y ;

4. FREE $\leftarrow$ FREE + 1;

RETURN;

END;

DF_OP	DF_X	DF_Y
Opcode	Operand 1 (X)	Operand 2 (Y)
1	30	30
2	50	80
2	70	30
1	40	55
2	60	55

Table 1.3 Display file structure

Command 1: 1, 30, 30

It moves current cursor position to point (30, 30)

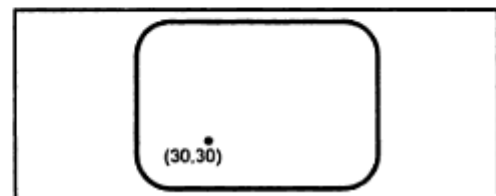


Fig. 1.25 (a)

Command 2: 2, 50, 80

It draws the line between point (50,80) and the current cursor position (30, 30)

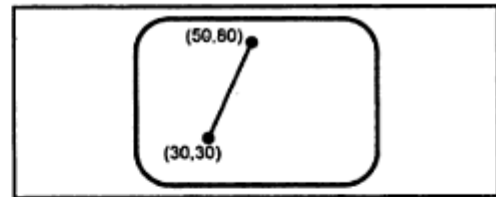


Fig. 1.25 (b)

Command 3: 2, 70, 30

It draws the line between point (70,30) and the current cursor position (50, 80)

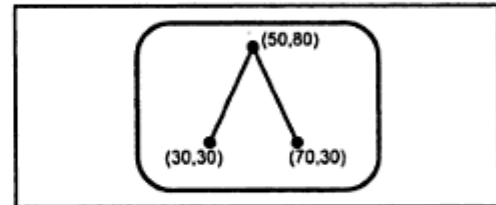


Fig. 1.25(c)

Command 4: 1, 40, 55

It moves current cursor position to point (40, 55)

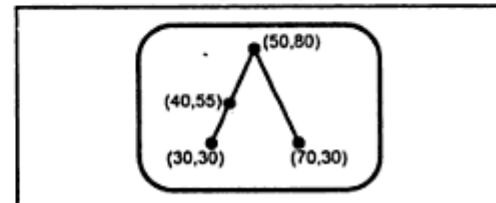


Fig. 1.25(d)

Command 5: 2, 60, 55

It draws the line between point (60,55) and the current cursor position (40, 55)

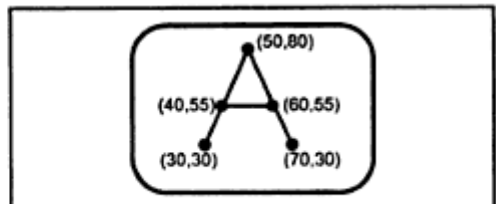


Fig. 1.25(e)